

Week 2 : DAY-3 Hands On - Keerthivasan R

LEVEL – 1 (Problem 1): Interactive Feedback Button

Scenario:

A startup company wants a simple interactive webpage where users can click a button to submit feedback and instantly see a confirmation message without refreshing the page.

Requirements

- Create a webpage with a "Submit Feedback" button.
- When clicked, display a confirmation message dynamically.
- Use JavaScript event listeners to handle the click event.

Technical Constraints

- Use plain HTML, CSS, and JavaScript.
- Event handling must use `addEventListener()`.
- No external frameworks allowed.

Expectations:

- Proper DOM manipulation.
- Clean and readable code.

Learning Outcome

- Understand browser event handling.
- Learn DOM manipulation basics.

CODE:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>FeedBack Trigger</title>
  <meta charset="UTF-8">
```

```
</head>
<style>
  body {
    font-family: Arial, Helvetica, sans-serif;
    background-color: #f4f4f4;
    text-align: center;
    padding-top: 100px;
  }

  textarea {
    width: 40%;
    height: 150px;
    padding: 15px;
    font-size: 18px;
    border-radius: 10px;
    border: 1px solid #aaa;
    resize: none;
  }

  button {
    padding: 10px 20px;
    font-size: 16px;
    cursor: pointer;
    background-color: #31af35;
    color: white;
    border: none;
    border-radius: 5px;
  }

  button:hover {
    background-color: #45a049;
  }

  button:disabled {
    background-color: #cccccc;
    cursor: not-allowed;
  }

  #message {
    margin-top: 20px;
    font-size: 18px;
  }

  .error {
    color: red;
  }

  .success {
    color: green;
  }
}
```

```

</style>

<body>

  <h2>Customer Feedback</h2>

  <textarea id="feedbackInput" placeholder="Enter Feedback"></textarea>
  <br>
  <button id="feedbackBtn">Submit Feedback</button>

  <div id="message"></div>

  <script>
    // Select the submit button element from the DOM
    // We store it in a variable so we can attach an event listener to it
    const button = document.getElementById("feedbackBtn");

    // Select the textarea where user types feedback
    // We need this to read user input value
    const feedbackInput = document.getElementById("feedbackInput");

    // Select the div where messages (error/success) will be shown
    // This allows us to dynamically update feedback messages
    const messageDiv = document.getElementById("message");

    // Add a click event listener to the button
    // This runs the function every time the button is clicked
    button.addEventListener("click", function () {

      // Get the value entered by the user
      // trim() removes extra spaces from beginning and end
      const feedbackText = feedbackInput.value.trim();

      // Check if the textarea is empty
      if (feedbackText === "") {

        // If empty → show error message
        messageDiv.textContent = "Please Enter Feedback!";

        // Apply CSS class to style the message as error
        messageDiv.classList = "error";
      }
      else {

        // If feedback is valid → show success message
        messageDiv.textContent = "☑ Thank you! Your feedback has been submitted successfully";

        // Apply success styling class
        messageDiv.classList = "success";
      }
    });
  </script>

```

```
        // Clear textarea after successful submission
        feedbackInput.value = "";
    }
});
</script>
</body>
</html>
```

OUTPUT:

Customer Feedback

Submit Feedback

Please Enter Feedback!

Customer Feedback

Submit Feedback

☒ Thank you! Your feedback has been submitted successfully

LEVEL – 1 (Problem 2): Theme Toggle Application

Scenario:

A small blog website wants to allow users to switch between Light Mode and Dark Mode interactively.

Requirements

- Create a toggle button to switch themes.
- Change background and text color dynamically.
- Store theme preference in localStorage.

Technical Constraints

- Use Vanilla JavaScript.
- Use addEventListener() for events.
- No CSS frameworks.

Expectations:

- Smooth theme switching.
- Proper usage of localStorage.
- Clean and modular JavaScript functions.

Learning Outcome

- Handling user interaction events.
- Managing browser storage.

CODE:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Theme Toggle App</title>
  <meta charset="UTF-8">
```

```

<style>
  body {
    font-family: Arial, Helvetica, sans-serif;
    padding: 40px;
    transition: background-color 0.3s ease, color 0.3s ease;
  }

  .light-mode {
    background-color: #ffffff;
    color: #000000;
  }

  .dark-mode {
    background-color: #1e1e1e;
    color: #ffffff;
  }

  button {
    padding: 8px 15px;
    cursor: pointer;
    font-size: 14px;
    margin-bottom: 20px;
  }
</style>
</head>

<body>

  <h2>Theme Toggle Application</h2>
  <button id="themeToggle">Switch Theme</button>

  <p>
    This a sample blog content area. The theme preference will ber saved
    even after refershing the page.
  </p>

  <script>
    // Select the theme toggle button
    // We need this to switch between light and dark mode
    const toggleBtn = document.getElementById("themeToggle");

    // Function to apply saved theme when page loads
    // This ensures theme preference remains after refresh
    const applySavedTheme = () => {

      // Get saved theme value from localStorage
      // localStorage stores data permanently in browser
      const savedTheme = localStorage.getItem("theme");

      if (savedTheme) {

```

```

        // If theme exists → apply that class to body
        document.body.classList.add(savedTheme);
    }
    else {

        // If no saved theme → default to light mode
        document.body.classList.add("light-mode");
    }
};

// Function to toggle between light and dark mode
const toggleTheme = () => {

    // Check if body currently has light-mode class
    if (document.body.classList.contains("light-mode")) {

        // Remove light mode
        document.body.classList.remove("light-mode");

        // Add dark mode
        document.body.classList.add("dark-mode");

        // Save dark mode preference in localStorage
        localStorage.setItem("theme", "dark-mode");
    }
    else {

        // Remove dark mode
        document.body.classList.remove("dark-mode");

        // Add light mode
        document.body.classList.add("light-mode");

        // Save light mode preference
        localStorage.setItem("theme", "light-mode");
    }
};

// Add click event listener to button
// So theme changes when button is clicked
toggleBtn.addEventListener("click", toggleTheme);

// Call function immediately when page loads
// This applies previously saved theme
applySavedTheme();
</script>
</body>
</html>

```

OUTPUT:

Theme Toggle Application

Switch Theme

This is a sample blog content area. The theme preference will be saved even after refreshing the page.

Theme Toggle Application

Switch Theme

This is a sample blog content area. The theme preference will be saved even after refreshing the page.

LEVEL – 2 (Problem 3): Dynamic To-Do List

Scenario:

A productivity application requires an interactive To-Do list where users can add, delete, and mark tasks as complete without refreshing the page.

📌 Requirements

- Add new tasks dynamically.
- Delete tasks on button click.
- Mark tasks as completed.
- Use event delegation for better performance.


```

    ul {
      margin-top: 20px;
      list-style: none;
      padding: 0;
      width: 300px;
    }

    li {
      background: white;
      margin-bottom: 8px;
      padding: 10px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      border: 1px solid #ccc;
    }

    .completed {
      text-decoration: line-through;
      color: gray;
    }

    .delete-btn {
      background-color: red;
      color: white;
      border: none;
      cursor: pointer;
      padding: 4px 8px;
    }
  </style>
</head>

<body>

  <h2>Dynamic To-Do List</h2>

  <input type="text" id="taskInput" placeholder="Enter task">
  <button id="addBtn">Add Task</button>

  <ul id="taskList"></ul>

  <script>

    // Create empty array to store tasks
    // Each task will be an object with text & completed status
    let tasks = [];

    // Select input, button, and list elements
    const taskInput = document.getElementById("taskInput");

```

```

const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");

// Function to add a task
const addTask = () => {

    // Get input value and remove extra spaces
    const text = taskInput.value.trim();

    // If input is empty → stop function
    if (!text) return;

    // Push new task object into array
    // completed: false means task is not done initially
    tasks.push({ text, completed: false });

    // Re-render task list after adding
    renderTasks();

    // Clear input field after adding task
    taskInput.value = "";
};

// Function to display tasks on screen
const renderTasks = () => {

    // Clear existing list before re-rendering
    taskList.innerHTML = "";

    // Loop through tasks array
    tasks.forEach((task, index) => {

        // Create new <li> element
        const li = document.createElement("li");

        // Store index as data attribute
        // Helps us identify which task was clicked
        li.dataset.index = index;

        // Insert task text and delete button inside li
        li.innerHTML = `
<span class="${task.completed ? "completed" : ""}">
    ${task.text}
</span>
<button class="delete-btn">Delete</button>
`;

        // Add li to ul
        taskList.appendChild(li);
    });
};

```

```

    });

    // Event delegation
    // Instead of adding event to each button separately
    taskList.addEventListener("click", (e) => {

        // Get task index from parent <li>
        const index = e.target.parentElement.dataset.index;

        // If delete button clicked
        if (e.target.classList.contains("delete-btn")) {

            // Remove task from array
            tasks.splice(index, 1);

            // Re-render updated list
            renderTasks();
        }
        // If task text clicked → toggle completed
        else if (e.target.tagName === "SPAN") {

            // Change completed status (true ↔ false)
            tasks[index].completed = !tasks[index].completed;

            // Re-render list
            renderTasks();
        }
    });

    // Add event listener to Add button
    addBtn.addEventListener("click", addTask);

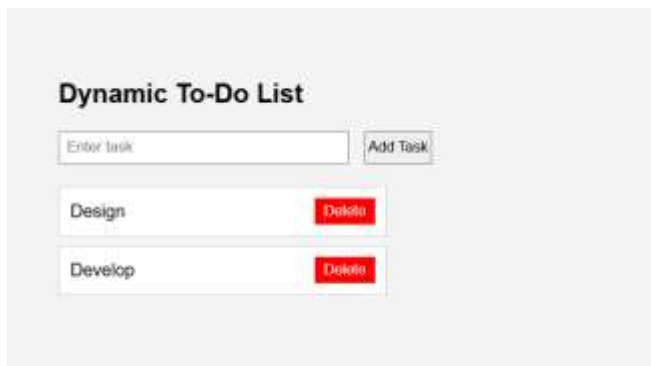
</script>

</body>

</html>

```

OUTPUT:



The screenshot shows a web interface for a 'Dynamic To-Do List'. At the top, there is a title 'Dynamic To-Do List'. Below the title, there is an input field with the placeholder text 'Enter task' and a button labeled 'Add Task'. Underneath the input field, there is a list of tasks. The first task is 'Design' and the second is 'Develop'. Each task is enclosed in a light gray box, and to the right of each task name is a red button labeled 'Delete'.

LEVEL – 2 (Problem 4): Mini Product Search App

Scenario:

An e-commerce company needs a mini product search application that filters products dynamically as users type in the search box.

📌 Requirements

- Display product list from a predefined array.
- Filter products dynamically using input event.
- Display “No Results Found” when applicable.

🔧 Technical Constraints

- Must use DOM manipulation.
- Event delegation required.
- Code should follow modular structure.

Expectations:

- Real-time filtering.
- Proper state management.

🎯 Learning Outcome

- State management basics.

CODE:

```
<!DOCTYPE html>
<html>

<head>
  <title>Mini Product Search</title>
  <style>
    body {
      font-family: Arial;
```

```

        background-color: #f4f4f4;
        padding: 40px;
    }

    h2 {
        margin-bottom: 20px;
    }

    input {
        padding: 6px;
        width: 250px;
    }

    ul {
        margin-top: 20px;
        list-style: none;
        padding: 0;
        width: 300px;
    }

    li {
        background: white;
        margin-bottom: 8px;
        padding: 10px;
        border: 1px solid #ccc;
    }

    .no-result {
        margin-top: 20px;
        color: red;
    }
</style>
</head>

<body>

    <h2>Mini Product Search</h2>

    <input type="text" id="searchInput" placeholder="Search product...">

    <ul id="productList"></ul>
    <p id="noResult" class="no-result"></p>

    <script>

        // Array of product names
        // This acts as our data source
        const products = [
            "Laptop",
            "Mobile",

```

```

    "Keyboard",
    "Mouse",
    "Monitor",
    "Headphones"
  ];

  // Select DOM elements
  const searchInput = document.getElementById("searchInput");
  const productList = document.getElementById("productList");
  const noResult = document.getElementById("noResult");

  // Function to display products
  const renderProducts = (filteredProducts) => {

    // Clear existing list before rendering
    productList.innerHTML = "";

    // If no products found
    if (filteredProducts.length === 0) {

      // Show "No Results" message
      noResult.textContent = "No Results Found";
      return;
    }

    // Clear message if results exist
    noResult.textContent = "";

    // Loop through filtered products
    filteredProducts.forEach(product => {

      // Create new list item
      const li = document.createElement("li");

      // Insert product name
      li.textContent = product;

      // Add to UI
      productList.appendChild(li);
    });
  };

  // Function to filter products based on keyword
  const filterProducts = (keyword) => {

    // Return products that include search keyword
    // toLowerCase() makes search case-insensitive
    return products.filter(product =>
      product.toLowerCase().includes(keyword.toLowerCase())
    );
  };

```

```

    });

    // Listen for typing inside search input
    searchInput.addEventListener("input", (e) => {

        // Get current input value
        const filtered = filterProducts(e.target.value);

        // Render filtered results
        renderProducts(filtered);
    });

    // Display all products when page loads
    renderProducts(products);

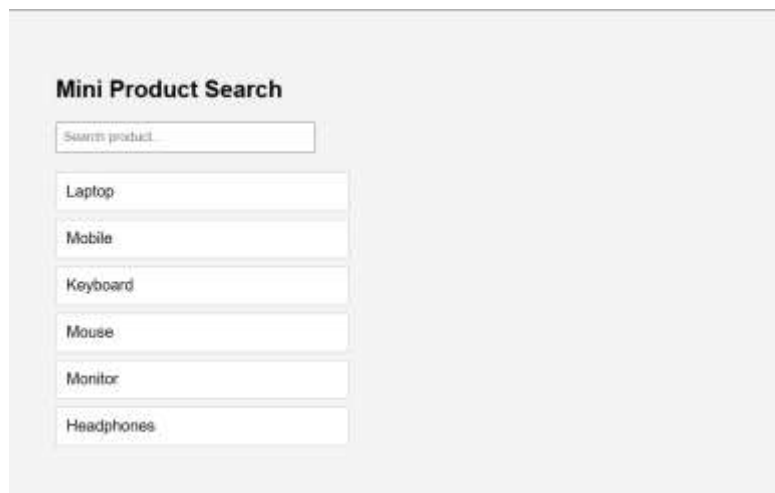
</script>

</body>

</html>

```

OUTPUT:



Mini Product Search

Search product

Laptop

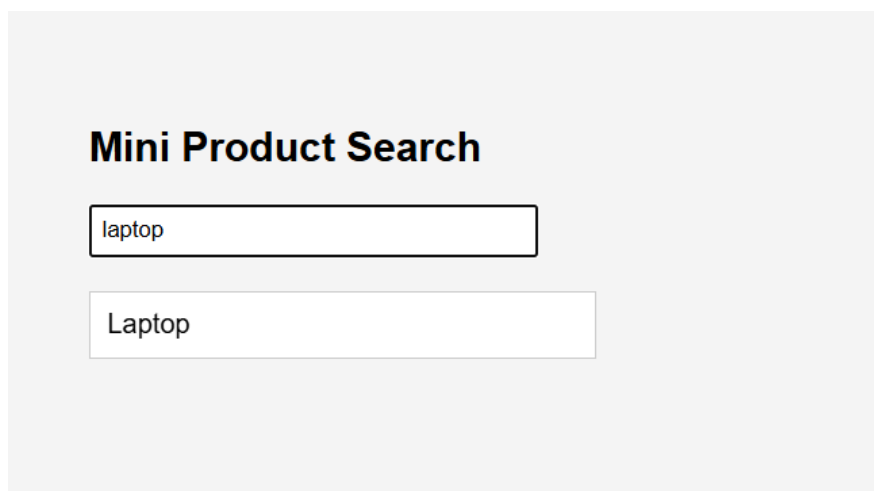
Mobile

Keyboard

Mouse

Monitor

Headphones



Mini Product Search

laptop

Laptop